Asterism: A Hebbian-Weighted Knowledge Graph as a Priority Index over Long-Term LLM Memory

Bidit Das
Independent Researcher, Plano, Texas, USA
orcid.org/0009-0009-3810-2946
github.com/biditdas18/asterism

Abstract

Large language model (LLM) assistants increasingly persist user information across sessions as unstructured "memory" summaries. Such summaries behave like an unindexed heap: every stored fact carries equal retrieval priority, and nothing signals which facts currently matter most. We present Asterism, a local-first system that builds a weighted, decaying knowledge graph from conversation history and injects a ranked subset into the model's context at inference time. The central design claim is architectural: the graph functions as a priority index over flat memory, in the same sense that a B-tree indexes an ordered file, with node weight acting as a salience signal analogous to an index key. We evaluate across five retrieval conditions, two independently authored personal graphs, 100 priority queries per graph, and five assistant models spanning four vendors, with decoding held fixed and the graph reseeded before every call. First, the deployed system — a weighted graph injected with a prioritization instruction — increases *decisiveness under ambiguity* over a flat list of the same facts: scored with a pre-registered, held-out-validated commit/hedge metric, every model commits to a weight-justified answer more often under the graph, in nearly every run. Second, that decisiveness is *more accurate* than flat memory in every model, but absolute accuracy remains low — the best model identifies the true top priority only about 40% of the time and most far less — so the graph delivers a strong relative lift on a problem far from solved. Third, we decompose the effect with two further conditions, the graph with its instruction stripped and a flat list with that instruction added. Decisiveness does not separate them: the instruction alone raises commitment in all ten model–graph cells (mean $+11.2$ points), by as much as the structure alone (mean $+10.7$), so a commit-rate gap is not by itself evidence of a structural effect. Accuracy does separate them. Structure alone improves correctness over a flat list in all nine measured cells, seven separably from zero, against five of ten for the instruction alone, and among committed answers it is more precise than the instruction-only condition in all nine measured cells ($+5.1$ to $+31.4$ points, four separable). The components are complementary rather than redundant: the deployed system's correctness gain matches or exceeds the better single component in all ten cells and strictly exceeds it in nine. A twelve-query pilot had read this decomposition as purely structural; at one hundred queries the supportable reading is that the instruction buys decisiveness while the weighted structure makes that decisiveness accurate. We report the instrument's own reliability (scorer–human agreement $\kappa = 0.64$, with errors that bias the measured gap toward the null), and two components an automatically-populated index requires: an entity-resolution layer whose accuracy ceiling we measure, and a supersession mechanism that demotes contradicted facts. Retrieval latency scales approximately linearly, near 34 ms at 10,000 nodes. We release the full system, both benchmark graphs, and all negative results.

1 Introduction

Conversational LLM systems now routinely retain information about a user between sessions. The dominant form this retention takes is a running natural-language summary: a list of facts, preferences, and history that is prepended to the model's context on later turns. This format is simple and effective for recall, but it is structurally flat. Every fact sits at the same level. There is no representation of which facts are central to the user and which are incidental, no representation of how facts relate, and no representation of which facts have been superseded by newer ones.

In database terms, a flat memory summary is an *unindexed heap*: a store you can scan but cannot query by priority. The contribution of this paper is to take that analogy literally and build the miss-

ing index. Asterism constructs a weighted directed graph from a user's conversation history, where nodes are concepts and entities, edges are relations, and edge and node weights encode how frequently and recently the user has engaged with each. At inference time, the highest-weight region of the graph is serialized into the model's context. The weight ordering is the index: it tells the retrieval step which facts to surface first, exactly as an index key tells a query planner where to look before touching the underlying data.

This framing makes a falsifiable prediction. If the graph acts as a priority index and not merely as a memory store, its benefit over an unstructured list of the same facts should appear specifically on tasks requiring prioritization and largely vanish on tasks requiring only recall. We test the prioritization half directly; the recall half is not tested (Limitations). The deployed graph condition does two things at once: it imposes a weighted structure *and* instructs the model to prioritize high-weight nodes. A naive graph-versus-flat comparison cannot separate these, so we run five conditions against the same underlying facts, two of which add the instruction without the structure and impose the structure without the instruction (Section 4). We hold decoding fixed, reseed the graph before every call so no response conditions on state a prior response wrote, and repeat the entire design across two independently authored personal graphs, one hundred priority queries per graph, and five assistant models spanning four vendors, five runs each.

We report three findings, each at its actual scope. First, on priority queries the weighted-graph-plus-instruction condition produces more decisive commitment than a flat list of the same facts in every model we test, over two graphs and one hundred queries each, in nearly every run. Second, that decisiveness is *more accurate* than flat memory in every model, but we state absolute levels plainly rather than only the relative lift: even the strongest model is correct only about 40% of the time and most well below that, so the graph improves priority routing without solving it. Third, we decompose the benefit into the graph *structure* and the prioritization *instruction*, correcting a reading a smaller pilot had encouraged. On decisiveness the two are indistinguishable, so a commit-rate gap cannot attribute the effect to structure; accuracy distinguishes them, with structure alone more often right than instruction alone in every measured cell, and the two act complementarily rather than redundantly. We prefer this decomposition, with its low absolute ceiling and underpowered cells stated openly, to the cleaner-sounding structural claim the full data does not support.

An index over a live, automatically-populated store raises two further engineering problems, which we treat as first-class. Entity resolution: without it the graph fragments into near-duplicate nodes that corrupt the weight signal. Supersession: a fact a later turn contradicts must be demoted, so a stale edge does not compete with the new one for retrieval. Sections 5 and 6 take up each in turn.

We make four contributions. We frame a weighted personal knowledge graph as a priority index over flat LLM memory, with node weight as the salience key (Section 3). We run a five-condition evaluation over two independently authored graphs, one hundred priority queries each, and five models from four vendors, which separates the deployed system's benefit from the value of memory per se and from the prioritization instruction (Section 4). We measure where embedding-based canonicalization of short concept labels stops working, and report that negative result (Section 5). We give a supersession mechanism that demotes contradicted facts from retrieval, with a latency study showing approximately linear scaling to 10,000 nodes (Sections 6, 7). All code, benchmarks, and the seeded evaluation graph are publicly available.

2 System Overview

Asterism is a local-first Python system. The entire graph lives in a single SQLite file on the user's machine; no data leaves the device except the user's own prompt and the injected context they choose to send to the model. Figure 1 shows the pipeline.

Extraction. Each conversational exchange is processed by a small extraction model that emits (source, relation, target) triples. Sources and targets become nodes; relations become labeled, weighted edges.

Weighting and decay. Every node and edge carries a scalar weight. Repeated engagement increases weight; disuse decreases it. Following the Hebbian principle that co-activated connections strengthen [1], an edge is reinforced whenever the model traverses it while answering a query. The complementary process is decay: an edge or node that accumulates session exposure without being traversed loses weight and is eventually pruned.

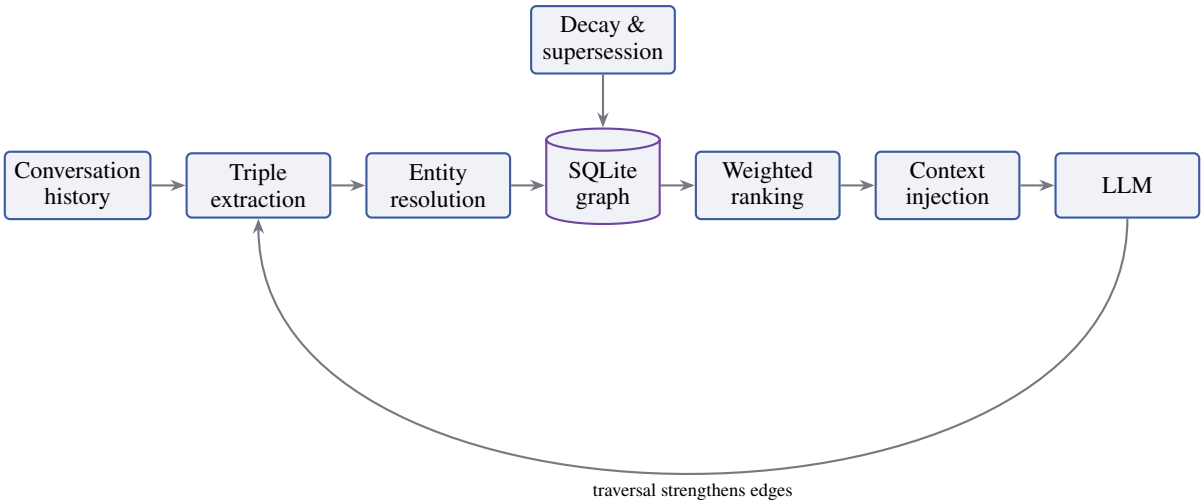


Figure 1: Asterism pipeline. Conversation history is extracted into (source, relation, target) triples, canonicalized by entity resolution, and written to a local SQLite graph. A background process applies decay and supersession. At inference time the highest-weight nodes are ranked and injected into the model's context. When the model traverses a relation in its response, the corresponding edge is strengthened, closing a Hebbian feedback loop.

Decay is measured in active session time rather than wall-clock time, so a concept does not fade while the user is simply away. This weight signal, accumulated over many sessions, is what the index ranks on.

Retrieval. On each turn, the graph is queried for its highest-weight nodes and their relations, which are serialized into a compact block and placed in the model's system context. This is the read path of the index. We deliberately keep it simple: an ordered scan by weight, returning the top-k nodes. Section 7 shows this is inexpensive even at ten thousand nodes.

3 The Graph as a Priority Index

The organizing idea of this work is that the structures a database uses to make an unindexed file queryable map onto the problem of making flat LLM memory prioritizable. We make the correspondence explicit because it drives both the design and the evaluation.

A flat memory summary is a heap file: an append-only list of facts with no access structure. You can read all of it, but you cannot ask it "what are the ten most important facts about this user" without scanning everything and imposing an ordering after the fact. A B-tree [2] solves the analogous problem for ordered files by maintaining a separate structure whose keys let a reader locate relevant records in logarithmic rather than linear cost. The key is not the data; it is a priority-ordered pointer into the data.

Asterism's weighted graph plays the role of that index over the heap of conversational facts. **Node weight corresponds to the index key:** weight is the quantity the retrieval step orders on, so a high-weight node is a high-priority pointer, surfaced first just as a B-tree directs a reader to the most relevant page before touching the file. **Graph traversal corresponds to query planning:** answering a query by walking weighted relations is a plan over the memory space rather than a full scan of the flat summary. **Decay corresponds to an eviction policy:** pruning low-weight, unused nodes keeps the working index small and current, so retrieval priority reflects present rather than historical relevance. And crucially, **the underlying facts persist** — decaying a node removes it from the *index*, not from the record of what was said, mirroring the removal of an index entry without deleting the row.

Graph-structured memory for LLMs is not itself new (Section 8). Our claim is narrower: that the *weighting* of such a graph should be understood and evaluated as an index key, and that doing so predicts where the structure helps. The next section tests that prediction.

4 Core Evaluation: Index Structure versus Flat Recall

4.1 Design

Evaluating a memory system risks conflating three different benefits: the benefit of having memory at all, the benefit of the particular structure imposed on it, and the benefit of instructing the model to act on that structure. The graph condition as deployed carries all three: it supplies facts, it imposes a weighted structure, and its system prompt instructs the model to prioritize high-weight nodes. To separate them, we run five conditions against the *same underlying facts*:

1. **Graph.** The thirty highest-weight nodes are injected in weight order, each labelled with weight and type, with an instruction to prioritize high-weight nodes (the deployed condition). No condition injects edges.
2. **Flat list.** Every node label is injected alphabetically and unweighted: the same facts are present, with the weight ordering and salience signal removed.
3. **Flat list + instruction.** The same unordered list, with the prioritize-and-commit instruction the graph carries added. This isolates the *instruction*.
4. **Graph, neutral.** The same weighted structure, with the explicit prioritize/commit instruction removed. This isolates the *structure*.
5. **None.** No memory is injected.

Two contrasts do the work. GRAPH vs. FLAT LIST isolates structure-and-weighting from information content, since both see identical facts. GRAPH, NEUTRAL vs. FLAT LIST, together with FLAT LIST + INSTRUCTION vs. FLAT LIST, separates the effect of the structure from the effect of the instruction. The pre-registered rule was: if the effect is instructional, the instruction added to a flat list should reproduce it and the instruction-free graph should lose it; if it is structural, the reverse. That rule presumes the two routes are mutually exclusive. They are not. On decisiveness both antecedents fire, so the dichotomy does not resolve on the commit metric (Section 4). We therefore resolve it on accuracy, which the same five conditions support with no change to the frozen scorer, the query set, or the labels. NONE is a reference point.

We evaluate on two independently authored personal graphs of roughly fifty nodes each, organized into weighted domain/theme/concept chains. The two describe deliberately different users: one an independent technical researcher, one a freelance designer with caregiving and family obligations. Their domain mixes and priority *shapes* differ, so a replication across both is evidence of generality rather than of one graph's idiosyncrasy. For each graph we focus the quantitative comparison on *priority queries*: one hundred questions that ask what the user should focus on, what matters most, or which of two things to prioritize, each requiring the model to choose or rank rather than merely recall. The queries were generated by filling each graph's node labels into a fixed bank of question templates, and were frozen before any evaluation call; because the generator is templated and not one of the models under test, the query set cannot favor any model. For each query we derive the ground-truth highest-priority node from the seed weights, and flag as near-ties those whose top-two candidates fall within the decay step. Near-ties are excluded from the correctness metric, since "correct" is ill-defined for them, but retained for the commit/hedge metric, which does not depend on ground truth.

To keep the comparison clean across a large call budget we fix three things. Decoding temperature is pinned to 1.0 on every backend, so cross-model differences cannot be attributed to sampling settings; we report the mean over five independent runs per cell rather than a single draw, and give run-to-run consistency for every contrast so that sampling variance is visible rather than averaged away. The graph is reseeded from the pristine seed before *every* individual call, so no response conditions on graph state that an earlier call's write-back mutated; an invariant check aborts a call rather than run it if the pre-call seed ever drifts. And we repeat the entire five-condition design across five assistant models spanning four vendors — `claude-sonnet-4-6`, `claude-opus-4-8`, `gpt-5.5-2026-04-23`, `deepseek-v4-pro`, and `kimi-k3` — with five independent runs each. These five are all token-billed frontier models with no hard daily request caps, which is what makes a one-hundred-query campaign feasible; four further models appeared in earlier pilot runs and were excluded on quota and throughput grounds, disclosed in full in Appendix D. The graph-maintenance extractor is a separate, fixed model held constant across all conditions (a local Llama 3.2 3B served over

Ollama, with an optional `claude-haiku-4-5` path). Concept-label embeddings come from `BAAI/bge-small-en-v1.5` [10], 384 dimensions, also held fixed. The assistant model is therefore the only moving part; because the graph is reseeded before every call, extractor output never enters the evaluated context.

4.2 Metric

We score each response with a single binary metric: does it COMMIT to a specific ranked answer, or HEDGE? A response commits if it names a single top priority or makes an explicit choice and justifies it; it hedges if it defers the decision to the user, enumerates options without selecting, or answers "it depends." The classifier is a deterministic rule, pre-registered and frozen before the evaluation run. We validated it on a held-out set of eleven hand-labeled responses drawn from earlier development runs, not from the evaluation set; it agreed on 11 of 11. We report the frozen classifier's output without post-hoc adjustment. Restricting the metric to priority queries is deliberate: commit-versus-hedge is only meaningful for questions that call for a ranking. Descriptive recall queries, for which a flat list is sufficient by construction, do not admit this metric and are excluded from it.

Because the entire dependent variable rests on this classifier, we also validate it against human judgment on the evaluation data itself, not only the development set. One author independently labeled all 36 responses of one full run (twelve queries across the three core conditions) as commit or hedge without seeing the classifier's labels: raw agreement was 83% and Cohen's κ was 0.64 (substantial). All six disagreements run in the direction that understates the graph condition relative to the flat list, so the measured graph$-$flat gap is a conservative estimate of the true difference; we did not retune the classifier after seeing them. Appendix D gives the breakdown, and the scorer–human comparison sheet is released with the code.

4.3 Findings

Table 1 reports the result across five models, two graphs, and one hundred queries per graph. We read it in three passes.

Finding 1: the deployed system increases decisiveness in every model. Compare the GRAPH column (the weighted graph plus prioritization instruction, as deployed) to FLAT LIST (the same facts, unweighted). The graph$-$flat gap (column G$-$F) is positive for all five models on both graphs, from $+2.6$ (Kimi G1) to $+38.8$ (Sonnet G2). It is also run-consistent: GRAPH beats FLAT LIST in all five runs for most model–graph pairs. The weakest case is Kimi graph 1 at 3/5, the one cell whose interval spans zero. NONE commits least throughout. Even GPT shows clear positive gaps here ($+15.2$, $+18.4$), though at pilot scale it had committed at a high baseline that masked any structural benefit. With a larger and harder query set, its no-memory rate is no longer a ceiling. The decisiveness benefit is not confined to models that hedge by default; it appears across the panel.

Finding 2: the decisiveness is more accurate than flat memory, but absolute accuracy is low. A metric rewarding commitment could reward confident error. Because each seed graph has known weights, the correct top-priority item is defined. We score correctness only on queries whose ground-truth weight separation exceeds the decay step, excluding near-ties, for which "correct" is not well defined. That leaves 96 of 100 scorable queries on graph 1 and 69 of 100 on graph 2, which has many near-ties by construction. We lead with the absolute numbers rather than the relative lift. Among scorable queries, the graph condition identifies the actually-highest-priority item in the following fractions: Sonnet 23%/30%, Opus 31%/21%, GPT 38%/42%, DeepSeek 9%/10%, Kimi 12%/11% (graph 1 / graph 2). So even the strongest model is correct on roughly four in ten priority queries, and most models on one to three in ten. Absolute priority routing is a hard, unsolved problem, and the weighted graph does not solve it. What it does is improve it: in every model and graph the graph condition is more often correct than the flat list (e.g. Sonnet 23% vs. 8%, Opus 31% vs. 8%, GPT 38% vs. 18%), so the added commitment is disproportionately correct commitment rather than confident noise. The finding is a real relative gain on top of a low absolute base, and we state both. The correctness gaps for all three contrasts, with intervals, two-sided p values, and run consistency, are the right-hand block of Table 1 and Appendix C; absolute correctness and precision for every condition are in Table 4.

Finding 3: decisiveness does not separate structure from instruction; accuracy does, and the two are complementary. Findings 1–2 concern the deployed system, which couples structure with

Model	Gr	Commit rate (%) None	Flat	Fl+in	Gr,neut	Graph	Commit-rate gap Fi−F	Gn−F	G−F	Correctness gap Fi−F	Gn−F	G−F
Sonnet	G1	0.8	26.0	32.4	45.4	47.8	**+6.4**	**+19.4**	**+21.8**	−0.2	**+14.2**	**+15.0**
Sonnet	G2	0.2	10.8	32.2	31.0	49.6	**+21.4**	**+20.2**	**+38.8**	**+8.7**	**+7.5**	**+24.1**
Opus	G1	1.0	13.2	30.8	36.8	49.2	**+17.6**	**+23.6**	**+36.0**	**+7.9**	**+12.9**	**+22.9**
Opus	G2	1.6	5.8	31.0	22.2	38.4	**+25.2**	**+16.4**	**+32.6**	**+11.3**	**+7.5**	**+17.7**
GPT	G1	23.6	35.6	40.4	40.8	50.8	+4.8	**+5.2**	**+15.2**	+0.4	**+13.3**	**+19.6**
GPT	G2	30.8	36.0	50.6	40.2	54.4	**+14.6**	+4.2	**+18.4**	**+8.7**	**+8.1**	**+16.5**
DeepSeek	G1	5.0	10.6	17.4	—	15.0	**+6.8**	—	**+4.4**	+2.9	—	**+6.2**
DeepSeek	G2	3.8	7.4	13.4	14.4	16.8	**+6.0**	**+7.0**	**+9.4**	**+5.2**	**+7.0**	**+7.0**
Kimi	G1	2.0	15.2	20.6	12.6	17.8	**+5.4**	−2.6	+2.6	+2.7	+0.8	+4.2
Kimi	G2	1.8	8.0	11.4	11.2	16.2	+3.4	+3.2	**+8.2**	+1.2	+3.8	**+6.1**

Table 1: Left: mean commit rate (of 100 priority queries) over five runs, by model, graph, and condition. Right: the three single-contrast gaps against FLAT LIST, on commit rate and on correctness per scorable query — **Fi−F** = instruction only (flat list plus instruction), **Gn−F** = structure only (graph with instruction removed), **G−F** = the deployed system (both). **Bold** marks a gap whose 95% percentile bootstrap interval excludes zero, paired by query over 10,000 resamples; the intervals, two-sided p values, and run consistency behind every bolded and unbolded entry are in Appendices B and C. Correctness is scored only on the queries whose ground-truth weight margin exceeds one decay step (96 of 100 on G1, 69 of 100 on G2); absolute correctness levels are in Table 4. Decoding pinned at temperature 1.0 on every backend, graph reseeded from a pristine seed before every call, all conditions scored by the same frozen held-out-validated classifier. DeepSeek's G1 GR,NEUT cell is omitted: the model returned recurring empty responses there (see text).

instruction. The design isolates each: GR,NEUT is the weighted structure with the instruction stripped, FL+IN the instruction without the structure. On decisiveness both work, by indistinguishable amounts. The instruction-only gap (Fi−F) is positive in all ten cells (mean $+11.2$ points, range $+3.4$ to $+25.2$) and separable from zero in eight. The structure-only gap (Gn−F) is positive in eight of nine measured cells (mean $+10.7$) and separable in six (Table 3, Figure 2). A twelve-query pilot had shown the structure-only route separating cleanly, and we read that as evidence the benefit was structural. At one hundred queries the reading does not survive: a bare instruction to prioritize and commit, carrying no weights and no structure, buys about as much decisiveness as the weighted graph does. A commit-rate gap cannot attribute the effect to structure, and we retract that stronger claim.

Accuracy separates them. Against the flat list, the structure-only condition improves correctness in all nine measured cells, separably from zero in seven. The instruction alone improves it in nine of ten cells but separably in only five, and it is flat in exactly the two cells where it moved decisiveness most (Sonnet G1 -0.2, GPT G1 $+0.4$). The sharper contrast is precision among committed answers — of the answers a model does commit to, what fraction are right. The structure-only condition is more precise than the instruction-only condition in all nine measured cells, by $+5.1$ to $+31.4$ points. That direction is unanimous, but only four of nine intervals exclude zero and run consistency is mixed (5/5 runs in three cells, 2/5 in one). We therefore report the sign pattern rather than per-cell effect sizes. Because cells share query sets across models, we attach no pooled significance statistic to the unanimity.

The components are complementary, not redundant. The deployed system's correctness gain matches or exceeds the better of its two single-component gains in all ten cells and strictly exceeds it in nine, and it is the only condition whose correctness gain excludes zero in nine of ten cells. The instruction gets a model to answer; the weighted structure is what makes the answer right; coupling them is worth more than either alone. Kimi graph 1, the weakest cell in the panel, is where nothing separates from zero on either metric, and DeepSeek's graph-1 structure-only cell is missing to the empty-response failure discussed below.

Qualitatively, no-memory responses hedge honestly ("I don't have your priorities"), while flat-list responses, holding the same facts, defer the choice back to the user. Hedging is defensible when no priority signal is present, so commitment measures resolution rather than correctness; Finding 2 supplies that check.

We record one model-specific failure, because the guard against it is part of the method. DeepSeek, in graph_neutral on graph 1, repeatedly returned responses reporting normal completion yet containing no text while still consuming out-

put tokens. A universal guard that refuses to score empty or truncated content caught every instance and aborted the affected cell rather than recording a spurious hedge, so we report that cell as missing; it caught an analogous failure in another model during development (Appendix C). Detecting and excluding such outputs, rather than silently scoring them as hedges, is a necessary property of automated LLM evaluation.

5 Entity Resolution and Its Ceiling

An index is only as good as the keys it orders. Because Asterism populates its graph automatically from extracted triples, the same concept arrives under many surface forms across sessions; without canonicalization "AI memory tools," "AI memory tooling," and "AI Memory Tools" become three nodes, each accumulating a fraction of the weight the underlying concept deserves. Fragmentation splits the salience signal and can prematurely decay a concept that is, in aggregate, highly active.

Two-stage resolution. We resolve labels at write time in two stages. A normalized exact/prefix match lowercases and whitespace-collapses labels and merges a new label that matches or forms a prefix of an existing one; this is cheap, precise, and caught every clear duplicate in our live label set. Because it cannot merge meaning-equivalent variants sharing no prefix, a second stage falls through to cosine similarity between label embeddings from a compact local model [10], merging above a calibrated threshold. The model runs offline on CPU on the write path only, so retrieval latency is unaffected.

The measured ceiling. The useful result is not that embedding resolution works but where it stops, measured against the live graph; the pattern held across every embedding model tested. Reworded or reordered labels reusing the same vocabulary score 0.92–0.98 and merge correctly ("CLI Tools"/"CLI tool" 0.982, "AI Memory Tools"/"AI memory tooling" 0.969). *True synonym substitution* is where it fails. "LLM requires GPUs" against "Large Language Models need graphics processors" scores only 0.745, inside the range of merely topically related pairs that must not merge ("Philosophy & Identity"/"Research Identity" 0.77–0.88). No single threshold separates the two kinds of pair. We calibrate to 0.92, below which false merges begin, making the resolver conservative: it eliminates rewording-driven fragmentation, the common case, while declining synonym pairs it cannot distinguish from distinct ones. Closing that gap needs supervision, either a fine-tuned equivalence model or an LLM semantic check, which we leave to future work.

6 Supersession: Demoting Contradicted Facts

An index over a live store must handle facts that change. When a user who "uses Postgres" switches to SQLite, naive decay eventually fades the old edge, but until it does both facts coexist and both can be retrieved as current. We add an explicit supersession mechanism scoped to the narrow, detectable case: a new edge from the same source, with the same relation, to a different target. On such a conflict the prior edge is marked superseded via a nullable self-reference and its time-to-live dropped so it becomes eligible for pruning; superseded edges are excluded from default injection, so the model sees only the current fact. A node is withheld only when *every* fact-edge to it is superseded, so a concept with any live relation still appears. Superseded edges are not deleted — they remain recoverable for explicit history queries ("what did I used to use"), preserving a distinction the flat summary cannot represent: a fact that is outdated versus one that was never true. The rule is deliberately conservative, avoiding open-ended contradiction detection that would risk demoting facts that only appear to conflict.

7 Latency and Scaling

For the graph to serve as an index without harming interactive use, the read path must stay cheap as the graph grows. We benchmarked the retrieval step (loading and weight-ordering nodes and edges for injection, excluding any model call) at three graph sizes over 20 runs each. Mean latency was 0.34 ms at 102 nodes, 2.81 ms at 1,006, and 33.87 ms at 10,051 (P95 38.7 ms). A $99\times$ node increase yields a $100\times$ latency increase, so retrieval scales approximately linearly. It stays under 34 ms at ten thousand nodes, negligible beside the model call it precedes. The linear rather than sub-linear factor is a full-table weight ordering, reducible with a secondary index on the weight column. The supersession filter adds 12–18% to read latency (30.1 to 35.2 ms at 10,051 nodes), a small cost of correctness within the interactive budget.

8 Related Work

LLM memory systems. MemGPT frames context management as a virtual memory hierarchy, paging information between a limited context window and external storage under LLM control [9]. Mem0 provides a memory architecture that extracts and consolidates salient facts from conversation, including a graph-structured variant [8]. Asterism shares the goal of persistent conversational memory but differs in emphasis: rather than optimizing recall or context-window management, it treats *prioritization* as the core function and evaluates the weighted graph specifically as an index that supplies salience. Our five-condition comparison separates that prioritization benefit from the recall benefit those systems primarily target, and separates the structure from the instruction to use it.

Importance-scored and graph-structured memory. The nearest prior art to a salience-weighted memory is the memory stream of generative agents, which retrieves observations by a weighted sum of recency, relevance, and an LLM-assigned importance score [3]; A-MEM similarly builds an evolving, interlinked note structure over agent memories [4]. Asterism differs in what the weight is and what is claimed for it. Weight accrues from repeated mention, decays on disuse, and attaches to graph nodes rather than episodes. We evaluate it as an *index key*: we test whether ordering the injected context by weight changes the assistant's answer, not whether an importance score improves retrieval recall. Structurally, GraphRAG builds entity graphs and community summaries for global queries [5], HippoRAG applies personalized PageRank over a knowledge graph for multi-hop retrieval [6], and Zep maintains a temporal knowledge graph with explicit fact invalidation [7], close in spirit to our supersession step (Section 6). Those systems optimize retrieval over a corpus or episode store. Our unit is a small personal graph, and our dependent variable is whether the assistant commits to a priority at all. Our contribution is the controlled decomposition of that effect into structural and instructional components.

Retrieval-augmented generation. RAG retrieves passages by similarity to a query [11]: it answers "what is relevant to this query," whereas Asterism's weighting answers "what is persistently important to this user," accumulated across sessions rather than computed per query. We take the empirical comparison between the two as the forward direction (Section 9).

Indexing and biological memory. The B-tree [2] is the canonical priority-ordered access structure over a file too large to scan; we borrow its framing directly as a design and evaluation lever. The weighting dynamics draw on Hebbian plasticity, in which co-active connections strengthen [1, 12], used as motivation for the traversal-strengthening rule, not a claim of biological fidelity.

9 Conclusion

We presented Asterism, a local-first system that builds a weighted, decaying knowledge graph from conversation history and uses it as a priority index over flat LLM memory. Across five conditions, two authored graphs, one hundred queries per graph, and five models from four vendors, the deployed system is more decisive than a flat list of the same facts in every model, and more accurate in every model. Absolute priority-routing accuracy nonetheless stays low, 42% at best and often far less, so the graph improves a hard problem without solving it. Decomposing that benefit, decisiveness does not separate the weighted structure from a bare instruction to prioritize: both raise commitment by comparable amounts, and we retract the stronger structural claim a twelve-query pilot had supported. Accuracy does separate them: the structure is more precise among committed answers in all nine measured cells. The components are also complementary, with the deployed pairing matching or exceeding the better single component in all ten cells. We also characterized entity resolution and supersession, showed retrieval scales approximately linearly to ten thousand nodes, and report the low absolute ceiling, the one weak model, and a model-specific empty-response failure our guard excluded.

The comparison these results motivate, which we do not frame as a contest, is against similarity retrieval. Similarity retrieval answers "what is relevant to this query"; a weighted decaying graph answers "what is persistently important to this user." The graph also adds a traceable, weight-grounded justification for why an item surfaced, and direct user control over what is remembered. Whether that paradigm beats a strong retrieval baseline on prioritization accuracy is worth answering either way, and it is where we take this next.

Limitations

Our core metric is a single binary outcome (commit vs. hedge), scored over one hundred priority queries per graph, five models, and five runs each. This is a substantial sample, but the queries are templated variations on a fixed bank of prioritization phrasings, so they span the intended task without spanning the full range of ways real users express priority. Two graphs is better than one but is still not a population of users; the two were authored to differ in domain and priority shape, yet both were authored by us, and author-constructed personas may share latent structure a real user distribution would not.

The system named in our title is not what the evaluation exercises. The weights in both benchmark graphs are authored, and the graph is reseeded from that pristine state before every call, so the reinforcement rule, the session-time decay, entity resolution and supersession never shape the evaluated context. What we measure is the index: whether a weight signal, once present, is usable as a ranking key. Whether the Hebbian loop produces a weight signal that tracks what a user actually treats as important is untested here, and it is the first thing a deployment study would have to establish. The strengthening rule is also ungated, in that an edge is reinforced whenever the model traverses it whether or not the answer was correct; Kairos [14] gates consolidation on a quality check for exactly this reason, and we did not test a gated variant. A further implication we do not test is the negative half of our own prediction (Section 3): that the graph's advantage should largely vanish on pure recall queries, which our commit/hedge metric cannot score.

Two limitations of the effect itself deserve emphasis. First, absolute accuracy is low: even the best model identifies the true top priority on only about 40% of scorable queries, and most models on 10–30%, so our claim is a relative improvement over flat memory, not a solved priority-routing problem. Second, the structure-only effect is not uniform, and which metric it is measured on matters. On *commit rate* the structure-only gap is separable from zero in only six of the nine measured cells — the interval includes zero for GPT graph 2 and both Kimi cells, and Kimi graph 1 is outright negative (-2.6 points) — which is part of why we do not rest the structural claim on decisiveness. On *correctness* the structure-only gap is positive in all nine measured cells and separable in seven, the two Kimi cells being the exceptions at $+0.8$ and $+3.8$ points; on precision among committed answers it is positive in all nine measured cells but separable in only four. Kimi is the weakest and least consistent model throughout, and DeepSeek's graph-1 structure-only cell is missing entirely because that model returned recurring empty responses in that condition. So the honest scope is that the correctness effect holds across nearly the whole panel, with one weak model, one missing cell, and several underpowered cells stated openly. Our controls isolate structure from instruction, but not the components within the structure, and the graph and flat arms differ in more than one way. The graph conditions inject the thirty highest-weight nodes with each weight printed; the flat conditions inject every node label alphabetically with no weights. The graph arm therefore receives the salience signal, a candidate set already pruned to the high-weight end, and an ordering, while the flat arm receives none of the three; all five prompts are reproduced verbatim in Appendix A. We did not run a weighted flat list, which would separate showing the weights from arranging them as a graph, nor a shuffled-weight condition holding topology fixed, which would separate the weights from the graph shape. What we show is that the deployed bundle beats an unranked list, the comparison a user actually faces, and not which element of the bundle is responsible. That the effect is bounded well below ceiling bears on this: a model handed an explicit ranking key identifies the top-weighted item on only 9–42% of scorable queries, so the contrast is not reducible to reading the answer off the prompt. Our panel also omits models we attempted but could not complete at this scale — two Gemini models (daily-quota-limited) and two free-tier open-weight models (throughput and task-comprehension failures) — which is a disclosed constraint of cross-vendor evaluation on consumer and free tiers.

We also do not compare end-task accuracy against a retrieval baseline such as RAG, or against external memory systems on a shared benchmark; the findings here are characterized behavioral effects on this setup, and the retrieval comparison is the forward direction outlined in the conclusion rather than a result we claim now. The entity-resolution ceiling is a limitation of the current unsupervised approach: true synonym pairs are not reliably merged and can still fragment the index. Supersession is scoped to the same-source/same-relation/different-target case and misses subtler

contradictions. Because decay is measured in active-session rather than wall-clock time, decay topology depends on usage frequency: a daily user prunes old interests quickly while a sparse user's graph decays little in real time and can accumulate stale structure over a year. Latency, while linear and small, is measured on synthetic graphs and on retrieval alone; end-to-end latency is dominated by the model call. Finally, the system is single-user and local by design, and we have not studied multi-user or privacy-preserving extensions.

Ethics Statement

Asterism is local-first: the graph is stored on the user's own device and no conversational data is transmitted except the user's prompt and the context they choose to inject into their model provider. Deleting the local store removes all derived data. The system infers user priorities from conversation history, and such inferences can be sensitive; because the data and index remain under the user's direct control and can be inspected or deleted, we regard the privacy surface as limited, but any deployment that centralized these graphs would raise concerns that the present design specifically avoids.

Declaration on Generative AI

The author used generative AI tools to assist with code for the system implementation and evaluation harness, and to assist with the language of the manuscript, by paraphrasing and polishing the author's own text for grammar, clarity, and organization rather than to generate new content. LLMs also appear as objects of study in the evaluation, as the assistant models being measured; those uses are described in the relevant sections. All research design, experimental decisions, analysis, and conclusions are the author's own, and the author reviewed and verified all content and takes full responsibility for it.

Acknowledgements

The author thanks the NORA 2026 reviewers, whose comments prompted the disclosures added to the Limitations section and the prompts reproduced in Appendix A. Generative AI tools were used in preparing this work: to assist with code for the system implementation and evaluation harness, and to assist with language editing of the author's own manuscript text. No text presenting research ideas, results, or conclusions was model-generated, and all research design, experimental decisions, analysis, and conclusions are the author's own. The Declaration on Generative AI above states this in full.

References

[1] Donald O. Hebb. 1949. *The Organization of Behavior: A Neuropsychological Theory*. Wiley, New York.

[2] Rudolf Bayer and Edward M. McCreight. 1972. Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3):173–189.

[3] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*.

[4] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[5] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.

[6] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[7] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*.

[8] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.

[9] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.

[10] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*.

[11] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe

Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474.

[12] Carla J. Shatz. 1992. The developing brain. *Scientific American*, 267(3):60–67.

[13] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. 2008. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference (SciPy2008)*, pages 11–15, Pasadena, CA.

[14] Pragya Singh and Stanley Yu. 2025. Validation-gated Hebbian learning for adaptive agent memory. In *Proceedings of the 1st Workshop on Knowledge Graphs & Agentic Systems Interplay (NORA 2025)*, volume 4162 of *CEUR Workshop Proceedings*, pages 42–54.

A Injected context for all five conditions

The system prompt is the only thing that differs across conditions; the user turn is the query verbatim and the assistant has no conversation history. In the two graph conditions $k = 30$: the thirty highest-weight nodes are injected, each rendered as `- {label} (weight: {w}, type: {t})` with the weight rounded to an integer. In the two flat conditions every node label is injected, alphabetically ordered, with no weight and no type. Both benchmark graphs hold roughly fifty nodes, so the graph conditions see a pruned candidate set and the flat conditions see all of it. **Relations are not injected in any condition**: the graph conditions receive a weighted node list, not a subgraph with edges, and the `TRAVERSAL` directive in the `graph` prompt only asks the model to label traversals it states in its own answer, which we parse for display. Table 2 reproduces all five system prompts verbatim. Long lines are wrapped there for the page; the strings themselves are single lines.

B Bootstrap Confidence Intervals on the Commit-Rate Gaps

The gaps in Table 1 are differences of means over a shared, frozen query set, so they admit a paired interval estimate without any additional model calls. For each model–graph cell we reduce every query to its commit rate across the five runs, pair each condition against FLAT_LIST by query identifier, and resample the one hundred query pairs with replacement 10,000 times (seed fixed at 20260902), reporting the 2.5th and 97.5th percentiles of the resampled mean difference and a two-sided bootstrap p-value. Pairing is at the query level rather than the run level because runs are exchangeable draws under identical decoding while queries are the unit the two conditions share. The frozen scorer is used as-is; no label was recomputed or reinterpreted for this analysis. The procedure is a single script in the repository (`research/bootstrap_ci.py`) that reads the committed per-cell result files and regenerates `n100_bootstrap_ci.csv` deterministically; p-values are floored at $1/B$, which is the resolution the resample count supports.

C Absolute Correctness, Precision, and Follow-up Intervals

Table 1 reports gaps; this section reports the absolute levels those gaps are computed from, and the interval estimates behind every bolded and unbolded entry. Correctness is the fraction of *scorable* queries (ground-truth weight margin exceeding one decay step: 96 of 100 on graph 1, 69 of 100 on graph 2) on which the condition committed to the actually-highest-priority node. Precision is the fraction of *committed* answers that were correct, with the number of committed answers in parentheses; it is the metric that separates the structural from the instructional component, and its denominators vary widely across conditions, so we print them. The NONE column's precision rests on single-digit denominators in most cells and should not be read as an estimate. Interval estimates use the same procedure as Appendix B — paired by query over 10,000 resamples of the frozen labels, no additional model calls, no relabelling — computed by `research/followup_ci.py`. That script draws its randomness from a generator seeded per row from the cell's own identifiers, so adding a contrast cannot perturb an interval already published in Table 3. Table 5 covers the instruction-only contrast on both metrics together with the structure-only correctness gap; Table 6 covers the deployed correctness gap and the precision difference between the two single components. Every table body in this paper is emitted from the committed result files by `research/make_paper_tables.py` rather than transcribed.

graph

```
You are Asterism, a personal knowledge assistant
for {user}.

You have access to {user}'s personal knowledge
graph. The following nodes represent their current
priorities and thought patterns, weighted by
recency and usage frequency:

ACTIVE NODES (highest weight first):
{node_lines}

When answering, reference these nodes naturally.
Prioritize information from high-weight nodes.
Note when you are drawing on specific parts of
their graph.
When you traverse a concept explicitly, format it
as: TRAVERSAL: NodeA -> NodeB
```

graph_neutral

```
You are Asterism, a personal knowledge assistant
for {user}.

You have access to {user}'s personal knowledge
graph. The following nodes represent their current
priorities and thought patterns, weighted by
recency and usage frequency:

ACTIVE NODES (highest weight first):
{node_lines}

Answer using this graph if relevant.
```

flat_list

```
You are Asterism, a personal knowledge assistant
for {user}.

Here are topics {user} has previously discussed,
in no particular order:

{node_lines}

Answer using this list if relevant.
```

flat_list_prioritized

```
You are Asterism, a personal knowledge assistant
for {user}.

Here are topics {user} has previously discussed,
in no particular order:

{node_lines}

When answering, reference these topics naturally.
Prioritize information from these topics. Note
when you are drawing on specific parts of this
list.
```

none

```
You are Asterism, a personal knowledge assistant
for {user}.

Answer using only the current conversation. You
have no access to {user}'s knowledge graph or
prior history.
```

Table 2: System prompts for all five retrieval conditions, verbatim. `{node_lines}` is the injected memory block: in the two graph conditions it is the thirty highest-weight nodes in weight order, each line `- {label} (weight: {w}, type: {t})`; in the two flat conditions it is every node label, alphabetical, with no weight and no type. No condition injects edges.

		Deployed: graph − flat_list					Structure-only: graph_neutral − flat_list				
Model	**Gr**	Gap	95% CI	p	Runs>0	SD	Gap	95% CI	p	Runs>0	SD
Sonnet	G1	+21.8	[15.2, 28.6]	<0.001	5/5	4.3	+19.4	[11.6, 27.6]	<0.001	5/5	3.9
Sonnet	G2	+38.8	[31.6, 45.8]	<0.001	5/5	4.9	+20.2	[13.8, 26.8]	<0.001	5/5	1.5
Opus	G1	+36.0	[30.4, 41.8]	<0.001	5/5	4.6	+23.6	[18.4, 28.8]	<0.001	5/5	5.8
Opus	G2	+32.6	[27.2, 38.0]	<0.001	5/5	6.4	+16.4	[11.0, 21.8]	<0.001	5/5	7.3
GPT	G1	+15.2	[9.4, 21.0]	<0.001	5/5	5.5	+5.2	[0.4, 10.0]	0.037	5/5	3.6
GPT	G2	+18.4	[13.4, 23.6]	<0.001	5/5	3.6	+4.2	[−1.4, 9.8]	0.146	4/5	5.4
DeepSeek	G1	+4.4	[0.6, 8.4]	0.025	4/5	4.7	—	—	—	—	—
DeepSeek	G2	+9.4	[5.2, 13.8]	<0.001	5/5	6.1	+7.0	[2.6, 11.4]	0.002	5/5	4.2
Kimi	G1	+2.6	[−3.0, 8.2]	0.369	3/5	6.0	−2.6	[−7.4, 2.2]	0.293	2/5	6.7
Kimi	G2	+8.2	[3.4, 13.2]	0.001	5/5	3.3	+3.2	[−0.6, 7.0]	0.119	5/5	0.4

Table 3: Paired bootstrap on the commit-rate gaps, in percentage points. RUNS>0 counts how many of the five runs show a positive gap; SD is the standard deviation of the gap across those five runs. The deployed gap excludes zero in nine of ten cells (Kimi graph 1 is the exception); the structure-only gap excludes zero in six of the nine measured cells, the exceptions being GPT graph 2 and both Kimi cells. DeepSeek's graph-1 structure-only cell was aborted by the empty-response guard and is not estimated.

D Reproducibility Details

Scorer–human agreement. Of the six disagreements in the 36-response validation run, the frozen classifier labeled as HEDGE four graph-condition responses a human read as commitments — for instance a bare imperative ("Ship the package") whose form the frozen rule's vocabulary did not cover — and over-credited two flat-list responses as commitments. Both directions understate the graph−flat gap, which is why we report it as a conservative estimate. We did not retune the classifier after seeing these disagreements, since doing so would fit the frozen scorer to the evaluation data it is used to score.

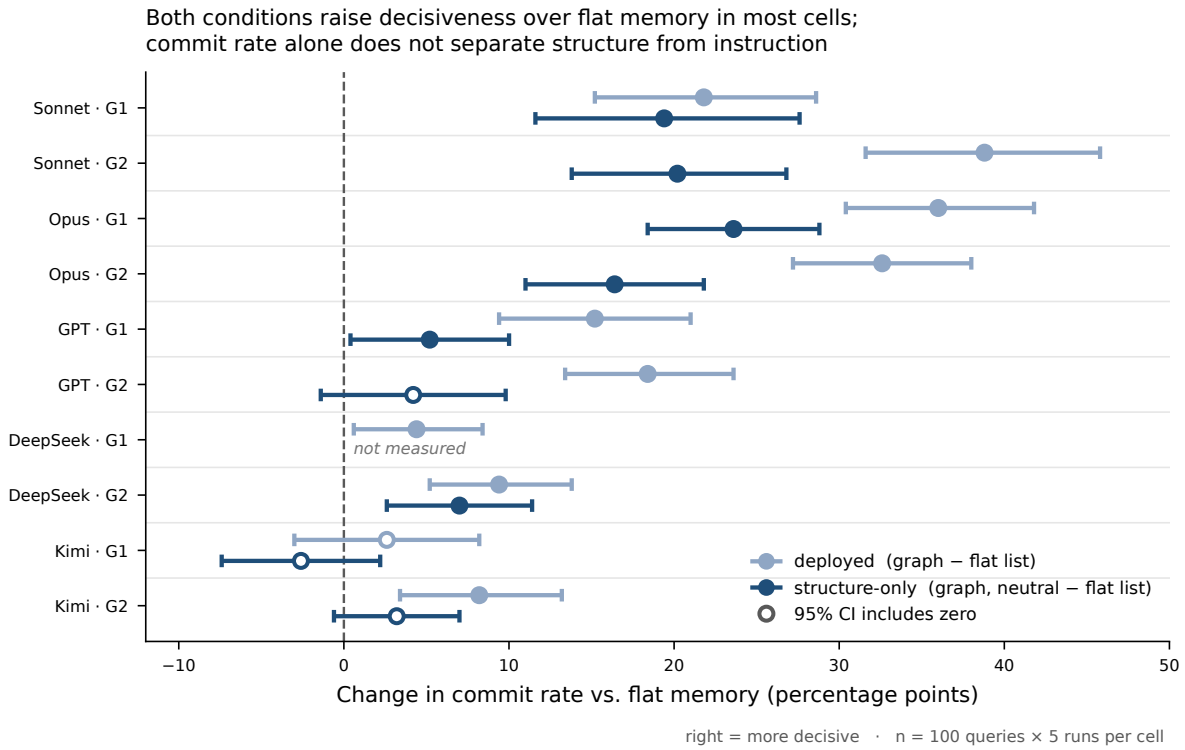


Figure 2: Change in commit rate relative to a flat list of the same facts, by model and graph, with 95% paired bootstrap confidence intervals. Filled markers denote intervals that exclude zero; open markers denote intervals that include it. The deployed contrast (graph plus prioritization instruction) is separated from the structure-only contrast (weighted graph, instruction removed) to show how much of the deployed benefit survives stripping the instruction. DeepSeek's graph-1 structure-only cell is annotated rather than plotted because the empty-response guard aborted it.

		Correct per scorable query (%)					**Precision among commits (%), (n committed)**				
Model	**Gr**	None	Flat	Fl+in	Gr,neut	Graph	None	Flat	Fl+in	Gr,neut	Graph
Sonnet	G1	0.0	8.3	8.1	22.5	23.3	0.0 (4)	32.0 (125)	25.3 (154)	50.9 (212)	48.9 (229)
Sonnet	G2	0.3	5.8	14.5	13.3	29.9	100.0 (1)	45.5 (44)	41.7 (120)	47.9 (96)	61.7 (167)
Opus	G1	0.0	8.1	16.0	21.0	31.0	0.0 (5)	60.0 (65)	51.7 (149)	56.7 (178)	63.1 (236)
Opus	G2	0.3	3.5	14.8	11.0	21.2	50.0 (2)	57.1 (21)	43.2 (118)	49.4 (77)	56.2 (130)
GPT	G1	7.9	18.3	18.8	31.7	37.9	34.9 (109)	52.7 (167)	47.4 (190)	78.8 (193)	76.2 (239)
GPT	G2	17.7	25.5	34.2	33.6	42.0	49.2 (124)	62.4 (141)	61.1 (193)	71.6 (162)	69.7 (208)
DeepSeek	G1	1.7	2.7	5.6	—	9.0	33.3 (24)	26.0 (50)	34.6 (78)	—	62.3 (69)
DeepSeek	G2	2.3	3.5	8.7	10.4	10.4	53.3 (15)	54.5 (22)	57.7 (52)	75.0 (48)	70.6 (51)
Kimi	G1	0.0	8.3	11.0	9.2	12.5	0.0 (10)	54.1 (74)	54.6 (97)	73.3 (60)	72.3 (83)
Kimi	G2	0.3	4.9	6.1	8.7	11.0	16.7 (6)	56.7 (30)	56.8 (37)	69.8 (43)	60.3 (63)

Table 4: Absolute correctness and precision by model, graph, and condition, the levels underlying the correctness gaps in Table 1. Correctness is over scorable queries (96 on graph 1, 69 on graph 2); precision is over committed answers only, with committed counts in parentheses, pooled over the five runs. Absolute correctness is low throughout — the best cell is GPT graph 2 at 42.0% — which is the ceiling we report rather than the relative gap alone. Precision is where the structure-only condition separates from the instruction-only condition: it is higher in all nine measured cells. NONE-column precision rests on denominators as small as one and is reported only for completeness.

Models excluded from the full-scale campaign. Earlier pilot runs also included two Gemini models and two free-tier open-weight models. The Gemini models' 250-request daily quota made a hundred-query, five-condition, two-graph campaign infeasible within the evaluation window. The two free-tier open-weight models failed both on throughput and on task comprehension, frequently naming the user node itself as the top priority. We therefore report the five token-billed models at full scale and exclude these four, which is a constraint of evaluating across vendors on consumer and free tiers

Model	Gr	Gap	95% CI	p	Runs>0	SD
Commit rate: FL+IN − FLAT *(instruction only)*						
Sonnet	G1	**+6.4**	[0.6, 12.2]	0.034	5/5	3.9
Sonnet	G2	**+21.4**	[16.0, 26.8]	<0.001	5/5	4.5
Opus	G1	**+17.6**	[12.6, 23.2]	<0.001	5/5	9.0
Opus	G2	**+25.2**	[20.6, 29.8]	<0.001	5/5	5.0
GPT	G1	+4.8	[−0.2, 9.8]	0.062	5/5	3.0
GPT	G2	**+14.6**	[9.2, 20.0]	<0.001	5/5	4.5
DeepSeek	G1	**+6.8**	[2.6, 11.0]	0.002	4/5	7.2
DeepSeek	G2	**+6.0**	[1.6, 10.6]	0.010	5/5	2.7
Kimi	G1	**+5.4**	[1.2, 9.4]	0.014	4/5	8.2
Kimi	G2	+3.4	[−0.2, 7.0]	0.066	4/5	3.6
Correct per scorable query: FL+IN − FLAT *(instruction only)*						
Sonnet	G1	−0.2	[−3.5, 3.1]	0.959	1/5	3.5
Sonnet	G2	**+8.7**	[4.1, 13.6]	<0.001	5/5	4.2
Opus	G1	**+7.9**	[3.8, 12.3]	<0.001	4/5	7.4
Opus	G2	**+11.3**	[6.4, 16.5]	<0.001	5/5	4.2
GPT	G1	+0.4	[−4.2, 5.0]	0.888	2/5	4.9
GPT	G2	**+8.7**	[2.9, 14.5]	0.005	5/5	4.7
DeepSeek	G1	+2.9	[0.0, 5.8]	0.054	4/5	2.3
DeepSeek	G2	**+5.2**	[2.0, 8.7]	<0.001	5/5	2.6
Kimi	G1	+2.7	[−0.6, 6.0]	0.111	4/5	2.4
Kimi	G2	+1.2	[−1.7, 4.1]	0.506	3/5	4.0
Correct per scorable query: GR,NEUT − FLAT *(structure only)*						
Sonnet	G1	**+14.2**	[9.2, 19.4]	<0.001	5/5	2.5
Sonnet	G2	**+7.5**	[2.9, 12.2]	0.003	4/5	6.9
Opus	G1	**+12.9**	[8.3, 17.7]	<0.001	5/5	7.0
Opus	G2	**+7.5**	[3.8, 11.9]	<0.001	5/5	5.5
GPT	G1	**+13.3**	[8.1, 18.8]	<0.001	5/5	5.2
GPT	G2	**+8.1**	[0.6, 15.4]	0.033	3/5	10.3
DeepSeek	G1	—	—	—	—	—
DeepSeek	G2	**+7.0**	[2.6, 11.9]	<0.001	4/5	6.1
Kimi	G1	+0.8	[−3.1, 4.8]	0.707	4/5	6.0
Kimi	G2	+3.8	[−0.3, 7.8]	0.077	4/5	2.6

Table 5: Paired bootstrap intervals, in percentage points, for the instruction-only contrast on both metrics and the structure-only contrast on correctness. RUNS>0 counts how many of the five runs show a positive gap; SD is the standard deviation across runs. **Bold** marks an interval excluding zero. The instruction alone raises the commit rate in all ten cells (eight separably), matching the structure-only commit gap of Table 3 in magnitude — which is why decisiveness cannot attribute the effect to structure. On correctness the instruction alone is separable in only five of ten cells, against seven of nine for the structure.

Model	Gr	Gap	95% CI	p	Runs>0	SD
Correct per scorable query: GRAPH − FLAT *(deployed)*						
Sonnet	G1	**+15.0**	[9.6, 20.4]	<0.001	5/5	5.5
Sonnet	G2	**+24.1**	[16.8, 31.6]	<0.001	5/5	3.3
Opus	G1	**+22.9**	[17.3, 28.8]	<0.001	5/5	4.5
Opus	G2	**+17.7**	[12.5, 23.2]	<0.001	5/5	4.0
GPT	G1	**+19.6**	[14.0, 25.4]	<0.001	5/5	6.2
GPT	G2	**+16.5**	[10.1, 23.5]	<0.001	5/5	3.3
DeepSeek	G1	**+6.2**	[3.1, 9.4]	<0.001	5/5	2.9
DeepSeek	G2	**+7.0**	[3.2, 11.0]	<0.001	5/5	3.1
Kimi	G1	+4.2	[−0.6, 9.2]	0.104	4/5	4.5
Kimi	G2	**+6.1**	[1.7, 10.7]	0.008	5/5	3.1
Precision among commits: GR,NEUT − FL+IN						
Sonnet	G1	**+25.6**	[13.6, 36.9]	<0.001	5/5	6.3
Sonnet	G2	+6.2	[−6.8, 19.3]	0.348	3/5	18.4
Opus	G1	+5.1	[−6.4, 16.7]	0.390	2/5	14.2
Opus	G2	+6.1	[−8.0, 21.1]	0.410	4/5	17.4
GPT	G1	**+31.4**	[22.7, 40.2]	<0.001	5/5	9.8
GPT	G2	**+10.5**	[0.6, 20.2]	0.039	4/5	8.7
DeepSeek	G1	—	—	—	—	—
DeepSeek	G2	+17.3	[−0.8, 33.7]	0.062	4/5	25.6
Kimi	G1	**+18.7**	[4.4, 31.8]	0.011	4/5	17.0
Kimi	G2	+13.0	[−7.1, 33.8]	0.204	3/5	32.2

Table 6: Paired bootstrap intervals, in percentage points, for the deployed correctness gap and for the precision difference between the two single components. The deployed correctness gap excludes zero in nine of ten cells, more than either component alone. The precision contrast is positive in all nine measured cells but separable in four, with run consistency between 2/5 and 5/5: the direction is unanimous while individual cells are underpowered, so we report the sign pattern and not per-cell effect sizes. No pooled statistic is given because cells share query sets across models and are therefore not independent.

rather than a property of the models' best available configurations.

The system is implemented in Python over SQLite, with the graph loaded into NetworkX [13] for analysis. Triple extraction uses a small instruction-tuned model; entity-resolution embeddings use a compact local sentence-embedding model [10] producing 384-dimensional vectors, run offline on CPU. The evaluation graph, the five-condition comparison script and its multi-model runner, the latency benchmark, and the entity-resolution and supersession test suites are included in the public repository, together with the frozen scorer, the scorer–human agreement sheet, and all per-run results; reproduction instructions are in the repository README (https://github.com/biditdas18/asterism). The exact state of the code behind every number in this paper is tagged `nora2026-camera-ready`: https://github.com/biditdas18/asterism/tree/nora2026-camera-ready. Latency figures were produced by timing the retrieval function directly with a monotonic clock over 20 runs per scale, on synthetic graphs seeded with weighted domain/theme/concept chains; the real user database is never touched by the benchmark.